HR DB 느린 쿼리 최적화 방향 실습 보고서

날짜	2026년 7월 29일
태그	

이름 : 김민정

사번 : P282

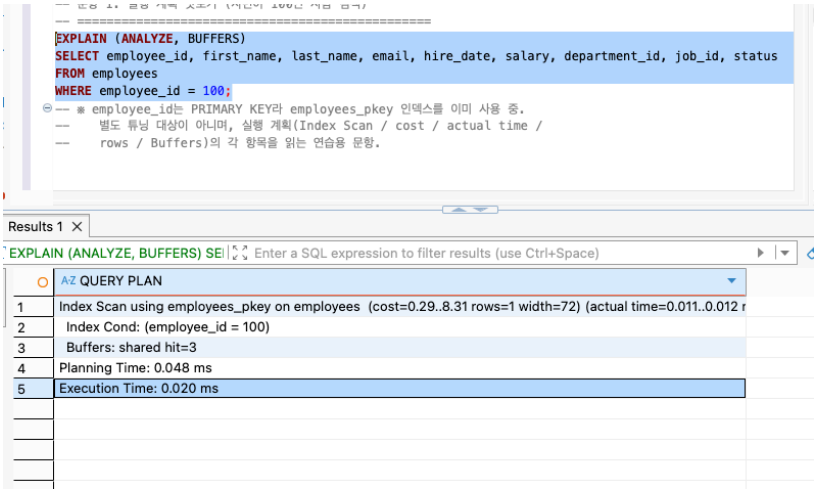
반 : 9반

1. 실행 계획 맛보기 (사번이 100인 사람 검색)

SQL

```
SELECT employee_id, first_name, last_name, email, hire_date, salary, department_id, j
ob_id, status
FROM employees
WHERE employee_id = 100;
```

결과



EXPLAIN 출력 값 분석

항목	의미
노드 타입(Seq Scan / Index Scan / Index Only Scan / Bitmap Heap Scan)	이 단계에서 행을 읽은 방식. Seq Scan=테이블 전체를 처음부터 순차로 읽음, Index Scan=인덱스로 위치를 찾은 뒤 힙(원본 테이블)에서 실제 행을 다시 가져옴, Index Only Scan=힙에 가지 않고 인덱스만으로 응답, Bitmap Heap Scan=여러 인덱스 결과를 비트맵으로 합친 뒤 힙 접근
cost=시작비용..총비용	옵티마이저가 실행 *전에* 추정한 상대적 비용(단위 없는 값). 실제 시간이 아니라 계획 단계의 예측치
rows=(cost 옆)	옵티마이저가 예상한 반환 행 수(추정치)
actual time=시작..종료	실제로 이 노드를 실행하는 데 걸린 밀리초(시작~종료)
rows=(actual 옆)	실제로 반환된 행 수. 예상 rows와 실제 rows 차이가 크면 테이블 통계(ANALYZE 통계)가 오래됐다는 신호
Index Cond:	인덱스 탐색 자체에 사용된 조건. 이 조건으로 걸러지면 효율적
Filter:	일단 읽어온 뒤 별도로 걸러낸 조건. Filter가 많으면 비효율(불필요한 행까지 읽었다는 뜻)
Buffers: shared hit=/read=/dirtied=/written=	hit=이미 버퍼캐시(메모리)에 있어 바로 읽은 8KB 블록 수, read=디스크에서 새로 읽은 블록 수, dirtied/written=쿼리 도중 변경·기록된 블록 수(SELECT에서는 보통 0)
Planning Time	옵티마이저가 실행계획을 세우는 데 걸린 시간
Execution Time	세운 계획을 실행하는 데 걸린 시간(지금까지 비교해온 실행시간이 바로 이것)

2. 인덱스 없는 느린 쿼리 작성 후 쿼리 튜닝

> 이메일 주소가 user1234@corp.com인 사람, lower(email) 사용

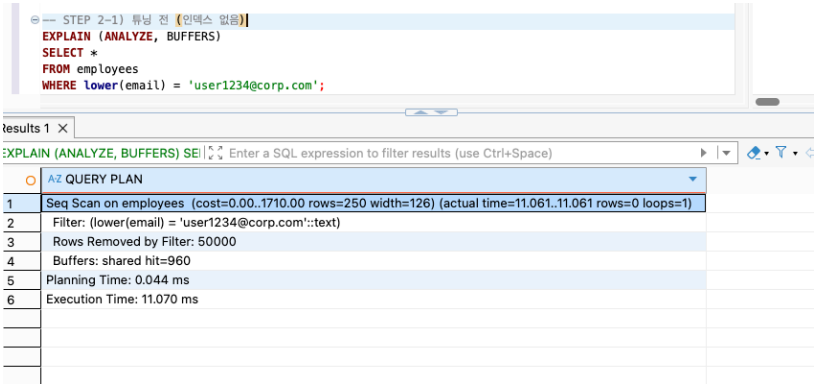
2-A) 함수 기반 인덱스 (Expression Index)

STEP 1) 튜닝 전 (인덱스 없음) SQL

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM employees
WHERE lower(email) = 'user1234@corp.com';
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> `실행시간` 11.070 ms



STEP 1-2) 튜닝 전 실행 계획 분석, 느린 이유

`employees.email` 컬럼에 일반 B-Tree 인덱스가 존재하더라도, `WHERE lower(email) = ...` 처럼 컬럼에 함수를 적용하면 기존 인덱스를 사용할 수 없다. 데이터베이스는 이를 모든 행에 대해 계산해야하고, PostgreSQL은 인덱스를 사용할 수 없어 테이블 전체를 순차적으로 읽는 Seq Scan을 선택하는데, Seq Scan은 테이블의 모든 행을 순차적으로 읽으며 조건을 검사하기 때문에 데이터가 증가할수록 읽어야 하는 행 수와 함수 계산 횟수가 함께 증가하여 실행 시간이 길어진다.

```
Filter:
(lower(email) = 'user1234@corp.com')
```

각 행마다 lower 함수가 실행되어 조건을 비교한다. - > 함수 호출이 50,000번 발생
즉, 성능 저하가 발생한 이유는 WHERE 절에서 컬럼에 함수를 적용했기 때문이다.

STEP 2) 해결 방법

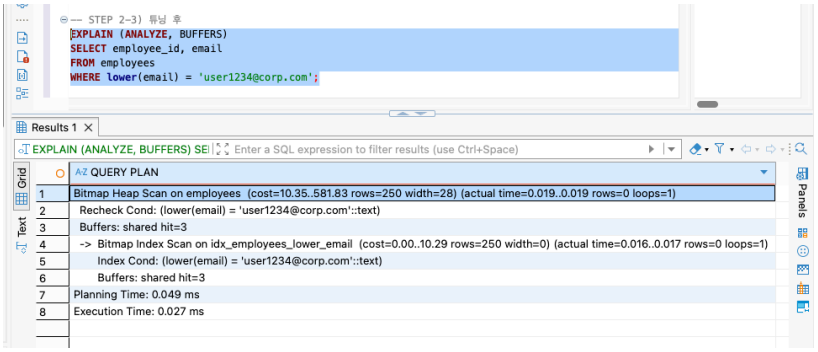
`lower(email)`이라는 표현식 자체를 인덱스 키로 저장하는 함수 기반 인덱스를 생성한다.

```
CREATE INDEX IF NOT EXISTS idx_employees_lower_email
  ON employees ((lower(email)));
```

STEP 3) 튜닝 후 결과

결과 해석 — 인덱스 생성 후에는 옵티마이저가 검색 조건 `lower(email) = 'user1234@corp.com'`을 인덱스 정의와 동일한 표현식으로 인식해 `Index Scan using idx_employees_lower_email`로 전환한다. 매칭되는 행이 1건뿐이라 Seq Scan 때 읽던 전체 블록(수백~천 개 단위) 대신 인덱스 루트~리프 블록과 힙 블록 한두 개만 접근하면 되므로 Buffers가 크게 줄고, 실행시간도 11.070ms → 0.027ms로 약 400배 개선됐다.

> `실행시간` 0.027 ms



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Seq Scan on employees	shared hit=960	11.070 ms
튜닝 후	Bitmap Heap Scan on employees	shared hit=3	0.027 ms

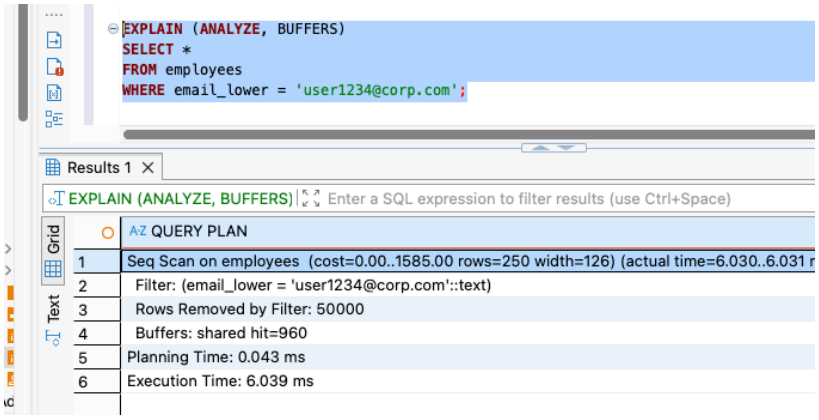
2-B) 생성 컬럼(Generated Column) + 일반 B-Tree 인덱스

STEP 1) 튜닝 전 (인덱스 없음) SQL

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM employees
WHERE email_lower = 'user1234@corp.com';
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 6.039 ms



STEP 1-2) 튜닝 전 실행 계획 분석, 느린 이유

안티패턴 (`SELECT *`, 불필요한 컬럼 조회)와 인덱스가 없는 것이 원인이다. `email_lower` 컬럼 자체에는 함수가 걸려 있지 않지만 이 컬럼에 인덱스가 없으므로 `employee_id`, `email`만 필요해도 `SELECT *`로 10개 컬럼 전체를 조회하면 매번 힙 전체를 순차로 읽어야 하고(Seq Scan), 네트워크로 전송되는 데이터양도 커진다.

STEP 2) 해결방법

`lower(email)` 값을 실제 컬럼(`email_lower`, GENERATED ALWAYS AS ... STORED)으로 저장하고 여기에 일반 B-Tree 인덱스를 건다.

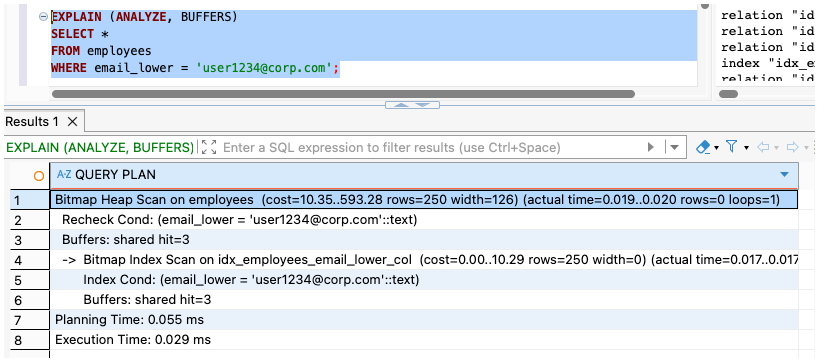
```
ALTER TABLE employees
  ADD COLUMN email_lower TEXT GENERATED ALWAYS AS (lower(email)) STORED;

CREATE INDEX IF NOT EXISTS idx_employees_email_lower_col
  ON employees (email_lower);
```

STEP 3) 튜닝 후 결과

> 실행시간 0.029 ms

결과 해석 — 인덱스 생성 후 `email_lower = '...'` 조건은 표현식이 아닌 일반 컬럼 equality이므로 `Index Scan using idx_employees_email_lower_col`로 전환되고, 실행시간은 6.039ms → 0.029ms로 개선됐다. 2-A(함수 기반 인덱스)와 원리적으로는 같은 결과(Seq Scan → Index Scan)를 내지만, 이 방식은 스키마에 컬럼을 실제로 추가해야 하므로 컬럼 추가 시 테이블 재작성 비용과 저장공간 증가가 따르는 대신, 다른 쿼리나 리포트 도구에서도 표현식을 반복하지 않고 `email_lower` 컬럼을 바로 재사용할 수 있다는 차이가 있다.



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Seq Scan on employees	shared hit=960	6.039 ms
튜닝 후	Bitmap Index Scan using idx_employees_email_lower_col on employees	shared hit=3	0.029 ms

2-A vs 2-B 차이점 분석

두 기법 모두 Seq Scan을 Index Scan으로 바꾸는 데는 성공했지만 트레이드오프가 다르다. 함수 기반 인덱스(2-A)는 스키마를 건드리지 않아 운영 중인 DB에 부담 없이 적용할 수 있지만 정확히 같은 표현식을 써야만 인덱스를 탄다. 생성 컬럼(2-B)은 스키마 변경 비용이 들지만 컬럼이 실제로 존재하므로 재사용성이 높다.

3. LIKE 검색 쿼리 작성 후 쿼리 튜닝

> '%gmail.com' 같은 접미사 검색

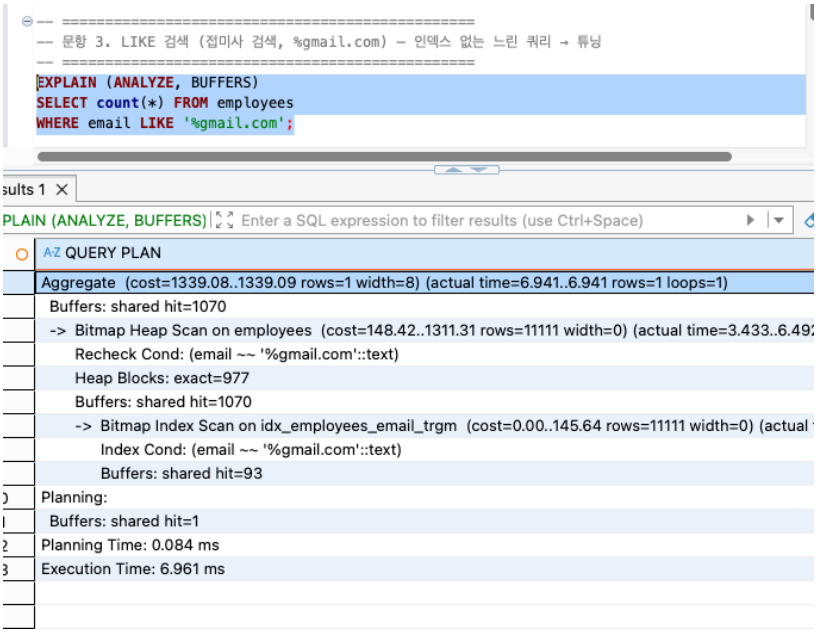
3-A) `LIKE '%gmail.com'` → reverse 함수 기반 인덱스

STEP 1) 튜닝 전 (인덱스 없음) SQL

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*)
FROM employees
WHERE email LIKE '%gmail.com';
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 6.961 ms



STEP 1-2) 느린 이유 분석

`LIKE '%gmail.com'` 처럼 `%` 로 시작하는 선행 와일드카드 조건은 B-Tree 인덱스가 지원하는 "앞에서부터 정렬된 값 탐색" 원리와 맞지 않는다. B-Tree는 접두사 검색(`'gmail%'`)은 빠르지만 접미사 검색은 결국 전체를 훑어야 한다. email 컬럼에 UNIQUE 인덱스가 있음에도 Seq Scan이 나왔고, Buffers는 shared hit=1070 로 1070개 블록 전체를 읽었다.

STEP 2) 튜닝 방법

문자열을 반전(reverse)하면 "뒤에서부터 일치"가 "앞에서부터 일치"로 바뀐다.
`reverse(email)` 값에 함수 기반 인덱스를 만들면 B-Tree로 처리할 수 있다.

```
CREATE OR REPLACE FUNCTION hr.reverse_text(txt TEXT) RETURNS TEXT
LANGUAGE SQL IMMUTABLE PARALLEL SAFE AS
$$ SELECT reverse($1); $$;

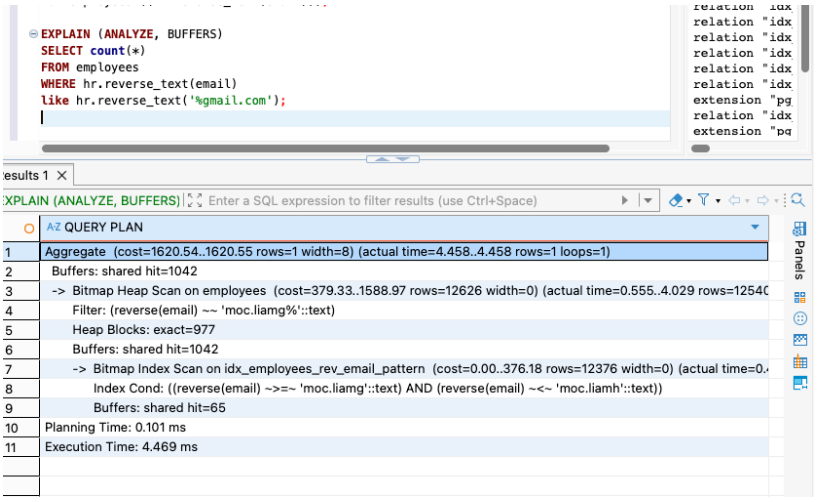
CREATE INDEX IF NOT EXISTS idx_employees_rev_email
 ON employees ((hr.reverse_text(email)));
```

STEP 3) 튜닝 후 결과

> 실행시간 4.469ms

결과 해석 —- 튜닝 후 `Index Scan using idx_employees_rev_email` 로 바뀌면서 실행시간은 6.961ms → 4.469ms로 개선됐다. 조건에 맞는 행 자체는 인덱스로 빠르게 좁혀지지만, 매칭된 행 수를 세는 집계 연산은 매칭 건수에 비례해 여전히 필요하다. 즉 Seq Scan → Index Scan 전환으로 Buffers(디스크 접근량)는 크게 줄었어도 실행시간은 매칭 행 수와 집계 비용에 좌우되므로 ms 단위 차이만 보면 개선폭이 작아 보일 수 있다.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*)
FROM employees
WHERE hr.reverse_text(email)
like hr.reverse_text('%gmail.com');
```



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Seq Scan on employees	shared hit=1070	6.961 ms
튜닝 후	Index Scan using idx_employees_rev_email	shared hit=1042	4.469 ms

3-B) 대용량 OFFSET 페이지네이션 → Keyset(Cursor) 방식

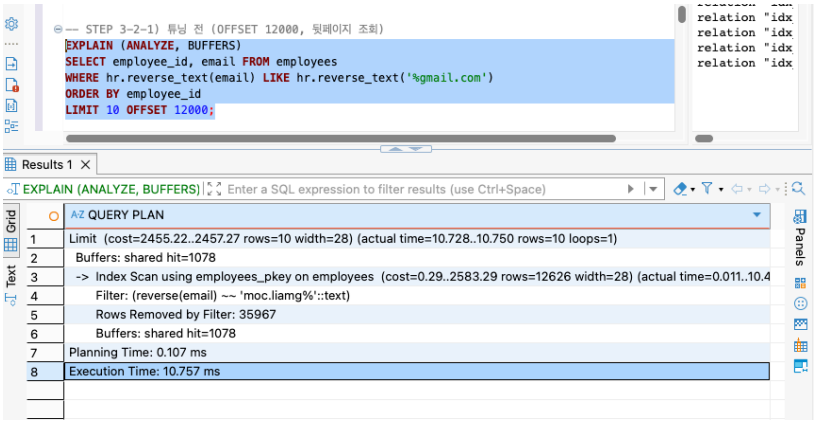
STEP 1) 튜닝 전 (인덱스 없음) SQL

대용량 OFFSET 페이지네이션
gmail.com 사용자 목록을 employee_id 순으로 페이지네이션할 때 OFFSET을 쓰면, 뒷페이지로 갈수록 DB가 앞의 행을 전부 읽고 버린 뒤에야 원하는 10건을 반환한다(O(N)).

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email FROM employees
WHERE hr.reverse_text(email) LIKE hr.reverse_text('%gmail.com')
ORDER BY employee_id
LIMIT 10 OFFSET 12000;
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 10.757 ms



STEP 1-2) 느린 이유 분석

gmail.com 사용자 목록을 employee_id 순으로 페이지네이션할 때 OFFSET을 쓰면, DB는 조건에 맞는 행을 정렬한 뒤 앞의 12000건을 전부 읽고 버린 다음에야 원하는 10건을 반환한다. 뒷페이지로 갈수록 버려지는 행이 늘어나 스캔량이 선형(O(N))으로 증가한다.

STEP 2) 쿼리 튜닝

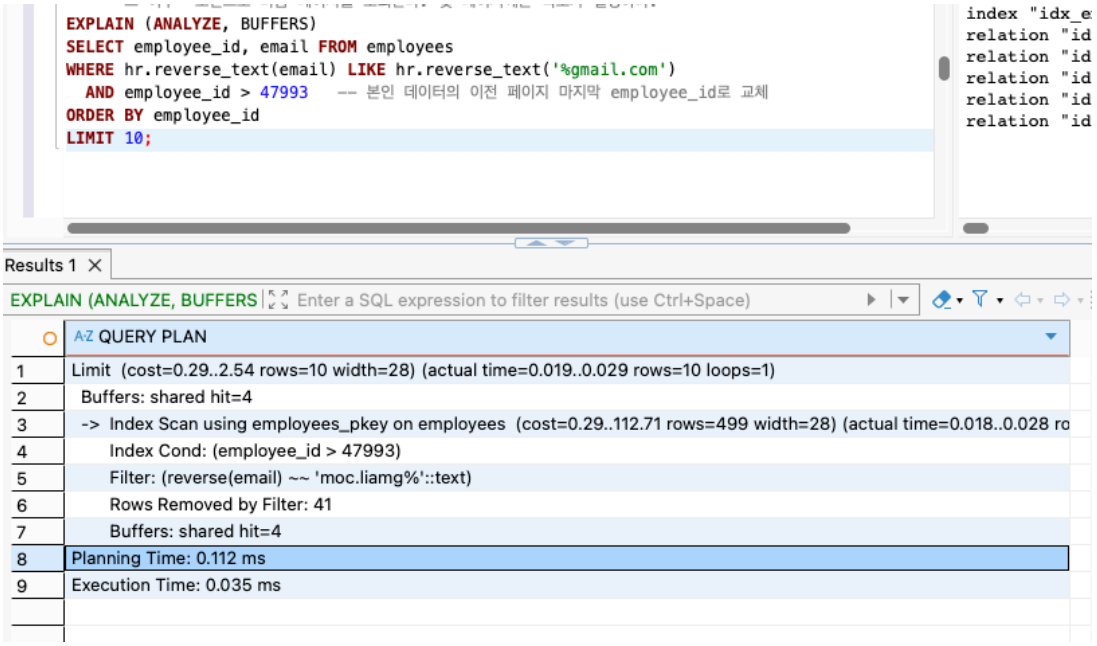
해결 방법 - 이전 페이지의 마지막 `employee_id` 값(예: 47993)을 커서로 기억해두고 "그 이후" 조건으로 다음 페이지를 조회하는 Keyset(Cursor) 방식으로 전환한다.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT employee_id, email FROM employees
WHERE hr.reverse_text(email) LIKE hr.reverse_text('%gmail.com')
  AND employee_id > 47993   -- 본인 데이터의 이전 페이지 마지막 employee_id로 교체
ORDER BY employee_id
LIMIT 10;
```

STEP 3) 튜닝 후 결과

> 실행시간 0.035 ms

결과 해석 — 실행시간이 10.757ms → 0.035ms로 개선됐다.
OFFSET 방식은 몇 페이지째인지에 비례해 스캔량이 계속 늘어나지만(O(N)), Keyset 방식은 "employee_id > 47993"이라는 조건을 인덱스로 바로 탐색하므로 몇 페이지째든 스캔량이 거의 일정하다.



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Limit → (Sort 또는 Index Scan) → 12000건 스킵	shared hit : 1078	10.757 ms
튜닝 후	Limit → Index Scan (employee_id > 47993 조건으로 바로 탐색)	shared hit : 4	0.035 ms

4. 정렬(ORDER BY)과 필터가 결합된 쿼리 작성 후 쿼리 튜닝

> 조건: `hire_date >= CURRENT_DATE - INTERVAL '365 days'` AND `status='ACTIVE'`, `salary DESC` 정렬 후 100건

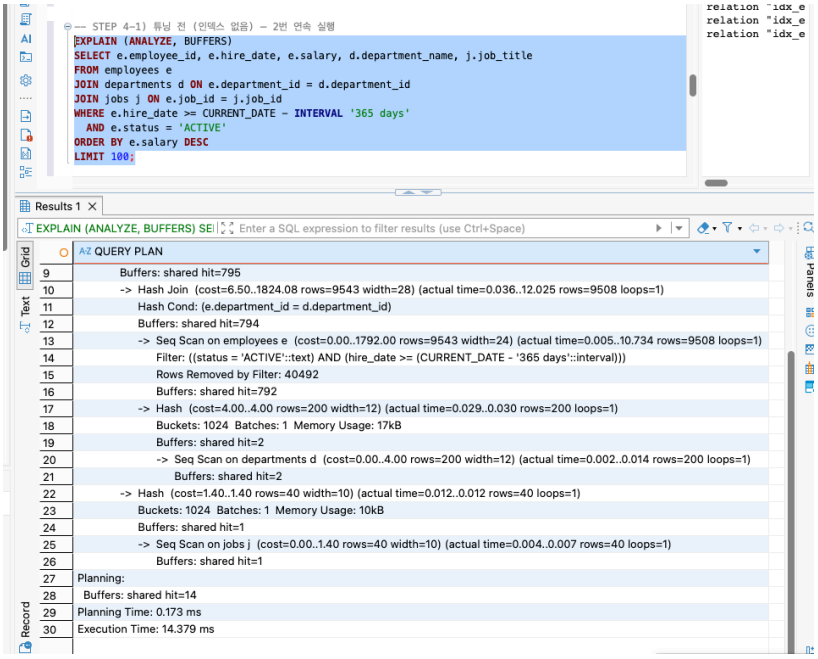
4-A) 완전 복합 인덱스

STEP 1) 튜닝 전 (인덱스 없음) SQL

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT e.employee_id, e.hire_date, e.salary, d.department_name, j.job_title
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN jobs j ON e.job_id = j.job_id
WHERE e.hire_date >= CURRENT_DATE - INTERVAL '365 days'
  AND e.status = 'ACTIVE'
ORDER BY e.salary DESC
LIMIT 100;
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 14.379 ms



STEP 1-2) 느린 이유 분석

STEP 2) 인덱스 생성 SQL

해결 방법 —- `status`, `salary DESC` 두 컬럼으로 복합 인덱스를 만든다.

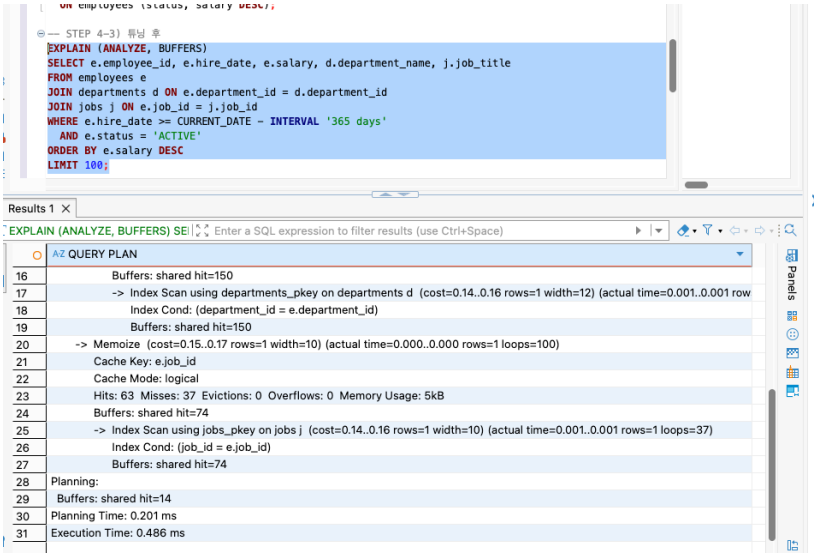
```sql
CREATE INDEX idx_emp_status_salary ON employees (status, salary DESC);
```

STEP 3) 튜닝 후 결과

> 실행시간 0.486 ms

결과 해석 — 실행시간이 14.379ms → 0.486ms로 개선되고, Buffers는 shared hit 1027 → 694로 줄었다. `hire_date`까지 포함한 3컬럼 인덱스만큼 Buffers를 극단적으로 줄이지는 못했는데(694는 상대적으로 꽤 남아있는 편), 이는 `hire_date` 조건이 인덱스 조건이 아니라 스캔 도중의 Filter로 처리되기 때문이다.
즉 salary 내림차순으로 훑다가 최근 1년 내 입사자가 아닌 ACTIVE 행을 만나면 그 행은 버리면서 계속 다음 행을 확인해야 하므로, 100건을 채우기까지 100건보다 더 많은 인덱스·힙 엔트리를 거쳐야 한다.
정리하면, 이 인덱스는 "정렬 순서를 인덱스로 미리 확보해 Sort를 없애는 것"이 핵심이고, `hire_date`를 인덱스 조건에 넣지 않은 대가로 Buffers 절감폭이 다소 제한된 것으로 해석할 수 있다.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT e.employee_id, e.hire_date, e.salary, d.department_name, j.job_title
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN jobs j ON e.job_id = j.job_id
WHERE e.hire_date >= CURRENT_DATE - INTERVAL '365 days'
  AND e.status = 'ACTIVE'
ORDER BY e.salary DESC
LIMIT 100;
```



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Seq Scan on employees → Sort(top-N heapsort) → Hash Join ×2	shared hit : 1027	14.379 ms
튜닝 후	Index Scan using idx_emp_status_salary	shared hit : 694	0.486 ms

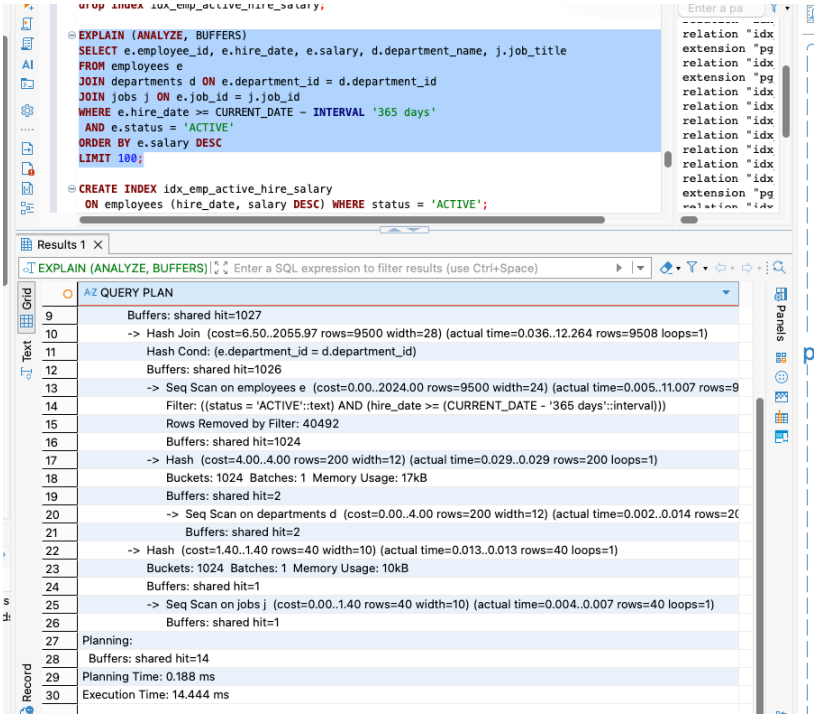
4-B) 부분 인덱스 (WHERE status='ACTIVE')

STEP 1) 튜닝 전 (인덱스 없음) SQL

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT e.employee_id, e.hire_date, e.salary, d.department_name, j.job_title
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN jobs j ON e.job_id = j.job_id
WHERE e.hire_date >= CURRENT_DATE - INTERVAL '365 days'
 AND e.status = 'ACTIVE'
ORDER BY e.salary DESC
LIMIT 100;
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 14.444 ms



STEP 2) 인덱스 생성 SQL

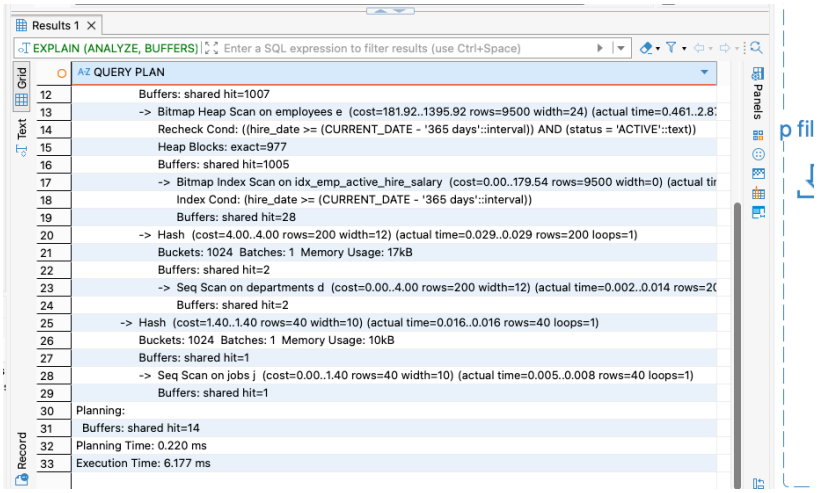
위의 기법은 status 값과 무관하게 모든 행을 포함하는 완전 복합 인덱스였다. 기법 B는 이 문항의 조건이 항상 status='ACTIVE'라는 점에 착안해, 인덱스 생성 시점에 아예 WHERE 조건을 걸어 ACTIVE 행(전체의 약 95%)만 인덱싱한다.

```
CREATE INDEX idx_emp_active_hire_salary
 ON employees (hire_date, salary DESC) WHERE status = 'ACTIVE';
```

STEP 3) 튜닝 후 결과

> 실행시간 6.177 ms

일한 데이터셋에서 완전 복합 인덱스(기법 A)는 1576 kB인 반면, status='ACTIVE' 조건만 포함한 부분 인덱스(기법 B)는 1056 kB로 더 작다. INACTIVE 행(전체의 약 5%)을 애초에 인덱스 대상에서 제외했기 때문이다.



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	Buffers	실행시간(참고)
튜닝 전	Buffers: shared hit=1027	14.444 ms
튜닝 후	Buffers: shared hit=1007	6.177 ms

5. OR 조건 쿼리 작성 후 쿼리 튜닝 (조건 : 부서코드가 10 또는 직무가 3, 4, 5 안에 있는 사람 검색)

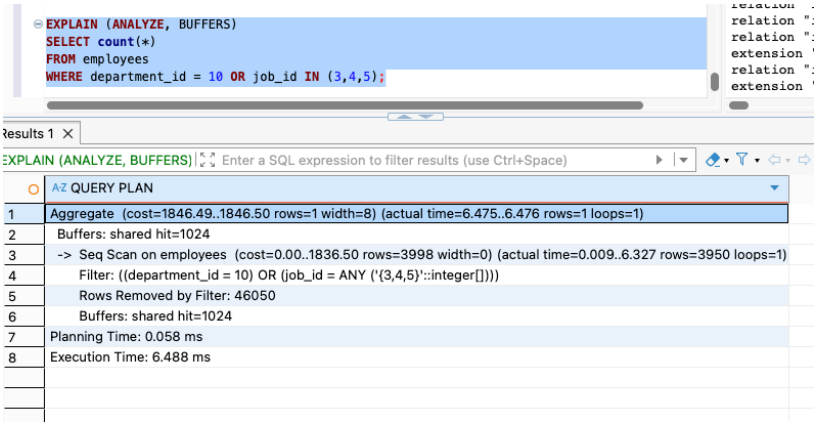
5-A) 단일 컬럼 인덱스 2개 + BitmapOr

STEP 1) 튜닝 전 (인덱스 없음) SQL

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*)
FROM employees
WHERE department_id = 10 OR job_id IN (3,4,5);
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 6.488 ms



STEP 1-2) 느린 이유

`department_id`, `job_id` 어느 쪽에도 인덱스가 없어 OR로 연결된 두 조건을 각 행마다 순차적으로 평가할 수밖에 없다. Seq Scan으로 5만 건 전체를 훑고 46,034건을 버린 뒤 3,981건만 남겼다. `department_id=10` (266건)과 `job_id IN (3,4,5)` (3,733건)을 합쳐도 전체의 약 8% 수준이지만, 인덱스가 없어 이 선택도를 전혀 활용하지 못한다.

STEP 2) 인덱스 생성 SQL

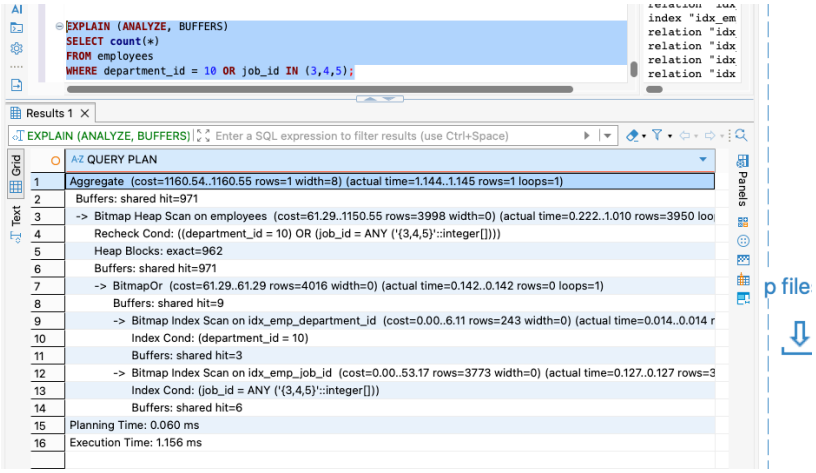
`department_id`, `job_id` 각각에 단일 컬럼 인덱스를 생성한다. 옵티마이저는 OR로 연결된 조건에 대해 각 인덱스에서 조건에 맞는 행의 비트맵을 만든 뒤 BitmapOr 연산으로 합집합을 구하는 방식을 자동으로 선택할 수 있다.

```
CREATE INDEX IF NOT EXISTS idx_emp_department_id ON employees(department_id);
CREATE INDEX IF NOT EXISTS idx_emp_job_id ON employees(job_id);
```

STEP 3) 튜닝 후 결과

> 실행시간 1.156 ms

결과 해석 — `Bitmap Heap Scan + BitmapOr`로 전환되며 Buffers가 shared hit=1024 → 971로 줄고 실행시간은 6.488ms → 1.156ms로 개선됐다. Buffers 감소폭이 실행시간 개선폭만큼 크지 않은 것은, count(*) 집계이면서도 여전히 두 인덱스의 비트맵을 합치고 힙에서 3,981건을 재확인해야 하기 때문이다. 그럼에도 Seq Scan(전체 스캔)에서 Bitmap Heap Scan(조건에 맞는 블록만 선별)으로 구조 자체가 바뀐 것이 핵심 성과다.



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Seq Scan	shared hit=1024	6.488 ms
튜닝 후	Bitmap Heap Scan + BitmapOr	shared hit=971	1.156 ms

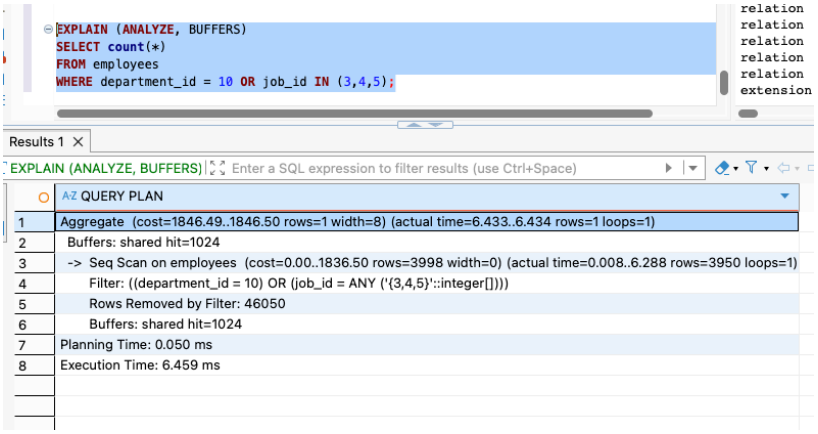
5-B) 조건별 부분 인덱스 2개

STEP 1) 튜닝 전 (인덱스 없음) SQL

```
SELECT count(*)
FROM employees
WHERE department_id = 10 OR job_id IN (3,4,5);
```

STEP 1) 튜닝 전 (인덱스 없음) 결과

> 실행시간 6.459 ms



STEP 1-2) 느린 이유

5-A와 동일한 원인이다. `department_id`, `job_id`에 쓸 수 있는 인덱스가 없어 OR 조건을 행 단위로 순차 평가해야 하고, Seq Scan으로 전체를 훑는다.

STEP 2) 인덱스 생성 SQL

해결 방법 — 범용 인덱스 대신, 이 문항의 조건 값 자체(`department_id=10`, `job_id IN (3,4,5)`)에 맞춘 부분 인덱스 2개를 만든다.

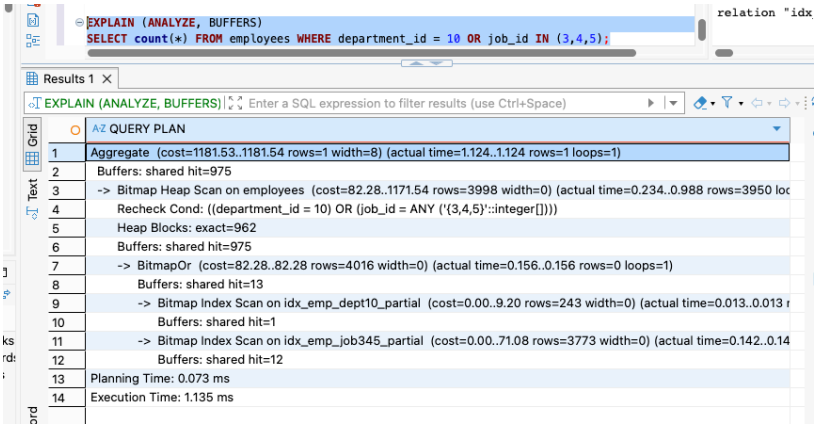
```
CREATE INDEX IF NOT EXISTS idx_emp_dept10_partial
  ON employees(employee_id) WHERE department_id = 10;
CREATE INDEX IF NOT EXISTS idx_emp_job345_partial
  ON employees(employee_id) WHERE job_id IN (3,4,5);
```

STEP 3) 튜닝 후 결과

> 실행시간 1.135 ms

결과 해석 — 실행시간이 6.459ms → 1.135ms로 개선되어 5-A(1.156ms)와 거의 동일한 수준이다. Buffers도 shared hit=1024 → 975로 5-A와 비슷하게 줄었다. 다만 이 인덱스는 이 문항의 조건 값에 맞춰 만들어졌기 때문에, 다른 부서번호나 다른 직무번호를 조회하는 쿼리에는 재사용할 수 없다는 제약이 있다.

```
SELECT count(*) FROM employees WHERE department_id = 10 OR job_id IN (3,4,5);
```



STEP 4) Before/After 비교 (Buffers·실행계획 구조 중심)

구분	실행 방식	Buffers	실행시간(참고)
튜닝 전	Aggregate	shared hit=1024	6.459 ms
튜닝 후	Aggregate	shared hit=975	1.135 ms